

vTen: Tensor-Centric Verification Framework for Domain-Specific Accelerators

Chanmin Baek, Keehyuk Lee, Mincheol Cha, Somi Hong, Xuan Truong Nguyen*, Hyuk-Jae Lee
 {bcm0314,lkh099,mccha}@capp.snu.ac.kr, ooihong0906@snu.ac.kr, {truongnx,hyuk_jae_lee}@capp.snu.ac.kr

Seoul National University
 Seoul, South Korea

Abstract

The semantic gap between tensor-centric software models and signal-level hardware testbenches creates significant productivity bottlenecks in verifying Domain-Specific Accelerators (DSAs). Existing frameworks like Cocotb suffer from prohibitive synchronization overheads due to fine-grained interactions. To address this, we propose vTen, a data-centric framework that strictly decouples verification intent from execution mechanics. By leveraging a declarative DSL and kernel-granular batching, vTen minimizes host-simulator interaction frequency. Evaluation on a production-scale 3D U-Net accelerator demonstrates that vTen achieves a 2× performance improvement in simulation latency and a 60.3% reduction in code complexity compared to Cocotb.

ACM Reference Format:

Chanmin Baek, Keehyuk Lee, Mincheol Cha, Somi Hong, Xuan Truong Nguyen[1], Hyuk-Jae Lee. 2026. vTen: Tensor-Centric Verification Framework for Domain-Specific Accelerators. In *63rd ACM/IEEE Design Automation Conference (DAC '26)*, July 26–29, 2026, Long Beach, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770743.3804323>

1 Introduction

Deep learning models have been widely adopted for various tasks such as image classification [8], object detection [18], and image segmentation [19]. The rapid popularity of diverse AI models is driving the development of domain-specific accelerators across academia [9, 14–16, 22] and industry [7, 11]. This workload diversity has driven the proliferation of domain-specific accelerators (DSAs) across the entire design stack—from RTL simulation and SoC integration to FPGA prototyping and ASIC tape-out. As a result, verification—the process of validating the correctness of hardware modules (e.g., register-transfer level (RTL) designs or FPGA implementation) against golden software workloads (e.g., written in PyTorch [17] or TensorFlow [2])—appears as an emerging challenge in hardware design development. Industry surveys report that verification consumes 60–70% of design effort [6].

The growing complexity of integrated circuits has intensified the challenges in hardware design verification. While SystemVerilog and Unified Verification Methodology (UVM) [1] are industry standards, their complexity raises entry barriers. For example, as

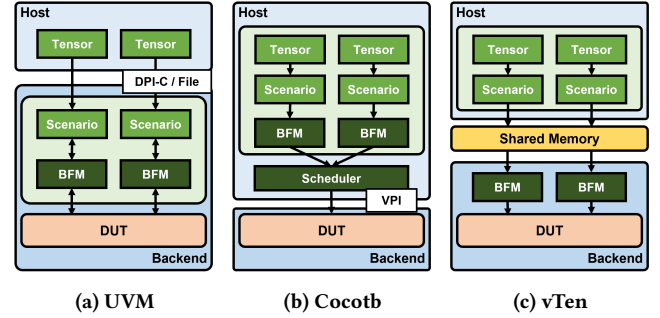


Figure 1: Comparison of Verification Frameworks: (a) UVM, (b) Cocotb, and (c) vTen.

illustrated in Figure 1(a), UVM requires converting a test vector, such as a Tensor, into a file that can be transferred to a simulation backend via Direct Programming Interface (DPI-C). However, UVM testing scenarios are handled at the backend, limiting its ability to handle diverse tensor data on the host or frontend. As verification is widely accepted as a software task, leveraging high-level languages such as C++ and Python has become a trend. Current hardware verification languages such as Python in Cocotb [20], and LuaJIT in Verilua [4] offer promising solutions to handle diverse cases in the frontend. For instance, as shown in Figure 1(b), testing scenarios are coupled with data tensors and a bus functional model (BFM) on the frontend, and the controls are transferred to the backend via a Verilog Procedural Interface (VPI). However, they may require transferring a large number of transactions via VPI, which typically results in slow simulation performance. Additionally, the approaches generally face poor component reusability across simulation environments.

A typical hardware verification flow involves translation across multiple levels, including tensors, bytes, transactions, and control signals, combining both data-centric and control-centric layers. As illustrated in Figure 1(c), a data-centric layer includes tensors and bytes that are primarily associated with data, its format, and testing scenarios for functional correctness verification on the software side. In contrast, a control-centric layer relates to the simulation backend, which primarily handles sending bus functional transactions and control signals on the hardware side. This observation poses a fundamental challenge in hardware design verification for DSAs, which is a *semantic gap*. Specifically, the essential goal of the data-centric layer differs from that of the control-centric layer. While the data-centric approach aims to verify the functional correctness of a hardware module by covering diverse test data, the control-centric layer primarily integrates simulation and waveform analysis. This suggests a strong message that an inappropriate coupling of them can be considered a root cause of productivity

¹Corresponding author: Xuan Truong Nguyen (truongnx@capp.snu.ac.kr).



bottlenecks and performance overheads in existing methodologies like [1, 20].

To overcome these challenges, we propose vTen, a data-centric verification framework that bridges a PyTorch-based tensor-centric dataflow and the hardware backend simulation. The main contributions of this work are as follows.

- **Methodology.** We propose a Data-Centric Abstraction that strictly decouples verification intent from execution mechanics. By combining a tensor-based Domain Specific Language (DSL) with Kernel-Granular Execution, vTen automates the translation of high-level dataflow into batched, protocol-specific transactions to minimize synchronization overhead.
- **Architecture.** To realize this methodology, vTen introduces a layered architecture that consists of a *Frontend* for defining verification scenarios using a DSL, a *Runtime* that lowers these scenarios into a backend-agnostic Execution Intermediate Representation, and a split-process *Backend* that executes batched operations via shared memory. This structure effectively bridges the semantic gap while maximizing simulation throughput.
- **Evaluation.** We validate vTen on a 3D CNN accelerator across diverse workloads, demonstrating: (1) 2× speedup in test execution compared to Cocotb through kernel-granular batching, (2) a 60.3% reduction in verification code complexity via the declarative DSL.

2 Background and Motivation

2.1 AI Domain-Specific Accelerators

Modern deep learning models typically consist of general matrix multiplication (GEMM) operations (e.g., convolution or fully connected layers) and non-GEMM operations, such as memory operations, normalization, and logit computation. Among them, GEMM operations are still dominant in terms of the number of operations and overall latency, ranging from 42.8% to 96.6% [12], and have been the prime optimization targets for domain-specific accelerators. The fundamental computing pattern of GEMM operations is the large use of basic operations such as multiplication and accumulation (MAC), which can be accelerated by massive parallel processing elements (PEs) [7, 9, 11, 14–16, 22].

To leverage highly parallel PEs, it is essential to consider massive data movements across various memory levels, including HBM or DRAM, cache or global memory, and registers [11, 14, 16]. For example, to execute a convolution layer, a DSA transfers weights and input feature maps (IFMs) from an external memory to a global on-chip buffer. In a DSA, data is moved between buffers and a PE array via a standard bus interface, such as AXI. The output results (so-called output feature maps (OFMs)) can be stored in on-chip memory or transferred to external memory via a direct memory access (DMA) module. In short, designing an AI accelerator requires managing massive data movements across different modules.

2.2 UVM and Cocotb

UVM [1] and Cocotb [20] present two essential approaches for hardware design verification. UVM is generally convenient for hardware designers by enabling low-level access and manipulation of test scenarios and parameters in the low-level hardware description

(a) UVM (Hybrid flow)

Data Gen: Python

```
# Same setup
fm = torch.randn(D, C, H, W)
packed_data = Pack(Quant(fm))
# Send to UVM via DPI/Socket
uvm_conn.send(packed_data)
```

↓ Cross-Language Transfer

Driver: SystemVerilog

```
// Native Execution Loop
task run_phase(uvm_phase phase)
;
forever begin
seq_item_port.get(req);

// Pin Wiggling (Fast)
vif.tdata <= req.data;
vif.tvalid <= 1'b1;

@(posedge vif.clk);
while(!vif.tready) @(
posedge vif.clk);
seq_item_port.item_done();
end
endtask
```

(b) Cocotb (Python-driven)

Data Gen: Python

```
# Same setup
fm = torch.randn(D, C, H, W)
packed_data = Pack(Quant(fm))
```

↓ Internal Function Call

Driver: Python

```
# High Overhead Loop
for word in packed_data:
# VPI Write
dut.tdata.value = word
# VPI Write
dut.tvalid.value = 1

# VPI Wait / Context Switch
await RisingEdge(dut.clk)
# VPI Read
while dut.tready.value ==
0:
# VPI Write
await RisingEdge(dut.clk)
```

Figure 2: An example of simulating a tensor with (a) UVM and (b) Cocotb.

language (HDL). Meanwhile, Python-based Cocotb is particularly appealing to computer architects who typically use cycle-level (CL) modeling methodologies [10]. Such CL-based approaches provide significant productivity benefits. For example, in a development flow with continuous integration, they allow for gradually replacing existing CL blocks with newly developed RTL designs in a large design, thanks to its abstraction.

2.3 Challenges for DSA Verification

DSA verification presents a unique challenge for existing methodologies such as UVM [1] and Cocotb [20]. As explained in Section 1, there exists a fundamental semantic gap between data-centric testbenches and control-centric hardware simulation during verification. The gap can be intensified when verifying hardware modules in a DSA design. The challenges can be described as follows.

Challenge 1 (Massive Repetitive Operations): To validate the functional correctness of a module in a DSA, it is generally necessary to conduct a simulation with large-scale tensors, for example, weights or IFMs of multiple convolution layers. As a result, it may impose considerable burdens on hardware designers when adopting UVM [1] for simulation/verification of large-scale tensors. For example, as illustrated in Figure 2(a), with UVM, the driving loop is offloaded to SystemVerilog. However, it requires complex cross-language data transfer, such as `uvm_conn.send(packed_data)`.

Meanwhile, from a control-centric simulation perspective, the flow is highly repetitive, consisting of a set of predefined operations. Here, as demonstrated in Figure 2(b), with Cocotb, the iteration occurs in the Python runtime (`for word in packed_data`). Consequently, every single data beat triggers multiple VPI operations: writing values (`dut.tdata.value`), scheduling callbacks (`await RisingEdge`), and reading status (`dut.tready`). For DNN workloads involving millions of cycles, this chatter between Python and the backend process creates a prohibitive performance bottleneck. **Challenge 2 (System Integration):** Designing a DSA also involves integrating and validating multiple block designs, including

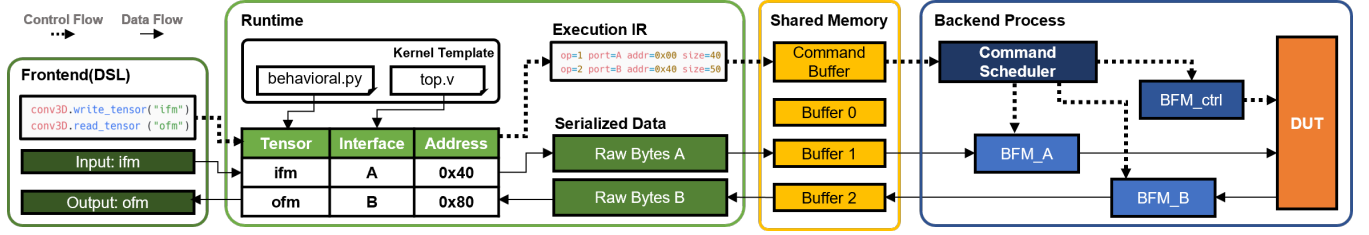


Figure 3: Overall Architecture of vTen.

massive-parallel PEs, global buffers, line buffers, DMAs, and controllers. In this context, UVM [1] suffers from a critical limitation in its ability to abstract block designs, especially for the verification of functional correctness. In contrast, using Cocotb [20] generally prolongs simulation time due to substantial VPI overhead at every sub-module in a large-scale design.

These observations pose a strong demand to design an alternative verification framework. A target design must effectively bridge the semantic gap between tensors and hardware simulation. It is also essential to achieve scalable performance for data-intensive workloads while achieving portability for design reuse and system integration.

3 vTen Architecture

3.1 Methodology and Architecture Overview

Methodology. vTen bridges the semantic gap and mitigates synchronization overheads via a tensor-centric approach, prioritizing throughput without sacrificing flexibility for diverse DSAs. The design rests on three fundamental principles: (1) *Data-Centric Abstraction* — decouples verification intent from execution mechanics by elevating the specification level from pin-level transactions to operations on tensors and registers; (2) *Kernel-Granular Execution* — treats the *Kernel*—defined as the accelerator’s top-level DUT encompassing compute logic and interfaces— as the atomic unit of interaction. By batching tensor operations into a single execution unit, it reduces host-simulator synchronization frequency from $O(C)$ to $O(1)$, where C denotes the kernel execution cycles; (3) *Shared Memory Data Plane* — utilizes serialized byte streams to enable direct backend access, bypassing the high-latency IPC overhead inherent in transaction-level interfaces.

Architecture. Following the design methodology, vTen introduces a layered architecture, as illustrated in Figure 3. Instead of a monolithic testbench, the framework decouples verification into three distinct layers, effectively separating high-level intent from low-level execution mechanics. First, at the **Frontend or Intent layer**, users define verification intent using a Python-based DSL. This layer focuses purely on dataflow, abstracting away cycle-level timing and physical port protocol. Second, at the **Runtime or Abstraction layer**, the runtime engine leverages Kernel Abstraction definitions to automatically lower high-level DSL operations into a backend-agnostic Execution Intermediate Representation (Execution IR) while handling tensor serialization and address mapping. Third, at the **Backend or Execution layer**, a split-process execution engine executes the compiled Execution IR batches. By utilizing shared memory and batch execution, the backend maximizes simulation throughput while reducing host-simulator transactions.

Table 1: Portable DSL Operations (Level 1)

Operation	Description
write_tensor	Transfers tensor data to DUT.
read_tensor	Retrieves tensor data from DUT.
write_register	Configures control register.
read_register	Queries current register state.
poll_register	Blocks until register satisfies predicate.
barrier	Enforces global synchronization.

The subsequent subsections detail the core components corresponding to these layers: Section 3.2 introduces the DSL for intent specification; Section 3.3 describes the Kernel Abstraction mechanisms managed by the runtime; and Section 3.4 explains the split-process Execution Layer that powers the backend.

3.2 Domain-Specific Language for Data-Centric Verification

3.2.1 Core Abstractions. The DSL serves as the declarative frontend interface, enabling users to define verification intent through high-level dataflow rather than low-level signaling. It provides a set of hardware-agnostic primitives that act as abstract templates for data movement (e.g., `write_tensor`, `read_register`) and logical synchronization (e.g., `barrier`). By abstracting away specific transport mechanisms (e.g., AXI4-Stream vs. AXI4-MM), these primitives strictly decouple the verification logic from interface implementation details. This allows the testbench to focus solely on the tensor-level dataflow intent, leaving the Runtime to automatically bind these abstract operations to the specific hardware protocols via Kernel Abstraction (Section 3.3).

3.2.2 Parallelism and Dependency Model. Hardware designs inherently exhibit massive spatial parallelism and non-blocking interactions. Expressing these concurrent behaviors within a sequential DSL syntax—without resorting to complex explicit constructs like SystemVerilog’s fork-join—presents a semantic challenge. To reconstruct this concurrency from a sequential DSL, vTen defines the operation lifecycle using two distinct events, *Issue* and *Commit*. Specifically, *Issue* marks the time stamp when an operation is dispatched to the DUT, while *Commit* marks the moment the DUT actually finishes execution and returns the result. The timing of these events is managed through the following two mechanisms:

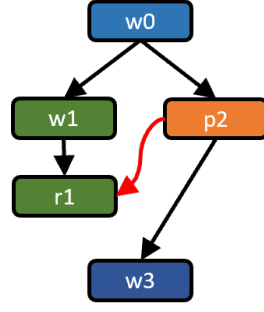
- (1) **Synchronization Mode (Implicit Control):** *Async* ($Issue_n \approx Issue_{n+1}$) allows immediate dispatch for parallel pipelines, whereas *Sync* enforces serial execution ($Commit_n \prec Issue_{n+1}$) by stalling the issue of the subsequent operation.
- (2) **Explicit Dependencies (Fine-grained Control):** Basic concurrency is refined by *Issue Dependencies* ($Commit_{src} \prec Issue_{dst}$) to resolve data hazards (e.g., RAW), and *Commit Dependencies*

```

1 # Layer N: Synchronization point
2 w0 = ctx.write_register("ctrl", 0x1)
3
4 # Branch 1: Process feature map 1
5 w1 = ctx.write_tensor("ifm_1", dep=w0)
6 r1 = ctx.read_tensor("ofm_1", dep=w1)
7
8 # Branch 2: Monitor completion
9 p2 = ctx.poll_register("done", dep=w0)
10
11 # Commit Dependency(r1, p2)
12 r1.add_dependency(p2)
13
14 # Layer N+1: Next synchronization
15 w3 = ctx.write_register("ctrl", dep=r1)

```

(a) DSL Specification



(b) Dataflow Graph

Figure 4: Example of mapping DSL code (a) and dataflow graph (b).

($Commit_{dst} \preceq Commit_{src}$) to model dynamic termination events like interrupts.

- (3) **Barrier Primitive (Global Synchronization):** To enforce coarse-grained ordering, the barrier primitive acts as a global fence. It stalls the dispatch of any subsequent operations until all in-flight operations reach their *Commit* state ($Commit_{prev} \prec Issue_{next}$).

3.2.3 Example: Parallel Feature Map Processing. Figure 4 demonstrates a fork-join pattern implemented via the DSL. The sequence initiates with a blocking operation w_0 (Line 2). Upon its completion, the control flow *diverges*: the primary data path executes the sequence $w_1 \rightarrow r_1$, while a background polling task p_2 is issued concurrently via *Async* mode. Line 12 establishes a *Commit Dependency* ($Commit_{r1} \preceq Commit_{p2}$), acting as a termination bound to preemptively terminate r_1 upon p_2 's completion. The two branches *converge* at the synchronization point w_3 (Line 15), restoring serial execution.

3.3 Bridging the Semantic Gap: Kernel Abstraction

To bridge the semantic gap, the **Kernel Abstraction** layer provides the necessary mapping intelligence to drive physical hardware ports from high-level tensor specifications. It automates the lowering of DSL operations into address-aware *Execution IR* and manages the data serialization/deserialization processes, thereby decoupling verification logic from architectural details. This abstraction is materialized through a two-phase lifecycle:

3.3.1 Build-time Kernel Template. The Kernel Template serves as a static, parameterizable blueprint of the Kernel. It is constructed by extracting interface specifications from lightweight RTL pragmas and retrieving tensor/register definitions from the Python-based functional reference model. Architectural attributes—including tensor shapes, register reset values, and protocol parameters—can remain parameterized at this stage. This separation allows verification logic to remain agnostic to specific architectural tuning (e.g., changing array sizes or bus widths).

3.3.2 Runtime Kernel Object. At runtime, the Kernel Template is instantiated into a concrete Kernel Object. The primary objective of this phase is to construct the *Binding Table*, which serves as the

(a) Interface Spec (RTL)

```

// vten:interface ifm_axi axi4_master
// vten:interface:parameter [addr: 64,
//   ↳ data: 256]
// vten:interface:addr_map ddr
// vten:interface:tensor ifm unsigned
//   ↳ [8]

```

(b) Functional.py

```

self.ifm = Tensor(
    shape=(1, "${C}", 32, 32),
    dtype=torch.float32)
def forward(self):
    return F.conv2d(self.ifm, self.
        ↳ weight)

```

Automated Binding

(c) Constructed Binding Table (Runtime)

```

TensorBinding(
    tensor_name="ifm", # Linked from functional model's tensor
    interface_name="ifm_axi", # Linked from RTL interface
    resolved_shape=(1, 64, 32, 32), # ${C} resolved to 64
    serializer=DataSerializer(...),
    address_mapping=AddressMapping(base=0x0, size=131072, addr_map="ddr")
)

```

Figure 5: Overview of the vTen compilation flow. Parameters and interfaces from RTL (a) and Python (b) are automatically resolved and bound in the runtime table (c).

definitive logical-to-physical map for execution, as illustrated in Figure 5. This is constructed through three consolidated steps.

First, the runtime concretizes the kernel configuration by resolving parameterized expressions within a unified namespace. By binding abstract attributes (e.g., $\${C}$) to target-specific values (e.g., $\{C: 64\}$), vTen enables seamless design space exploration across different accelerator variants without code modification. Second, the runtime establishes a bidirectional pipeline to bridge the software-hardware format gap. For serialization, the pipeline executes a three-stage transformation: (i) *Layout Transformation* to match the hardware memory hierarchy; (ii) *Quantization* to hardware fixed-point formats; and (iii) *Bit-packing* into bus-aligned byte streams. This infrastructure also supports the inverse deserialization flow, enabling the automated reconstruction of high-level tensors from raw hardware outputs. Finally, the runtime performs physical address allocation based on the size of the serialized byte streams and the interface address map. These resolved physical constraints are populated into the *Binding Table*, enabling the execution engine to resolve any tensor access with $O(1)$ lookup overhead.

3.3.3 Execution IR Lowering and Portability. Using the constructed *Binding Table*, the runtime translates DSL primitives into Execution IR—a backend-agnostic instruction format. For instance, a `write_tensor` DSL primitive is compiled into a generic command: `WRITE(interface_id, buffer_id, protocol, phys_addr, dep)`. This intermediate representation completely decouples verification intent from the backend execution layer. While current backends map Execution IR to VPI/DPI calls for simulation, the architecture is explicitly designed to support hardware emulation platforms (e.g., FPGA). The shared memory transport model (detailed in Section 3.4) maps naturally to PCIe DMA, allowing future backends to execute the same instructions on FPGAs, ensuring test portability across the verification lifecycle.

3.4 Scalable Execution Layer

We leverage the Backend Execution Engine (Figure 3) to implement a scalable execution strategy. This architecture is designed around

two core principles: minimizing runtime overhead through build-time code generation and eliminating synchronization bottlenecks via decoupled data/control transport planes.

3.4.1 Lightweight Runtime Architecture. To ensure the Backend Interface operates with minimal latency, vTen enforces a strict separation between static definition and dynamic execution. Computationally expensive tasks such as interface parsing and BFM code generation are offloaded to the *Build-Time* phase. Consequently, the *Runtime* execution loop becomes extremely lightweight, handling only $O(1)$ *Binding Table* lookups and dispatching pre-compiled Execution IR. This design eliminates recompilation overhead, facilitating rapid iterative testing on the target hardware.

3.4.2 Split-Process Execution Model. To overcome the synchronization bottlenecks identified in Section 2, vTen redefines the abstraction boundary between the host and the backend. Instead of tight cycle-level coupling, we decouple execution into two distinct planes: a Data plane and a Control plane.

Data Plane: Shared Memory Transport. Serialized tensors are pre-populated into a shared memory data plane prior to kernel execution. This design isolates data transport costs from simulation logic and enables direct backend access without IPC serialization overhead. Crucially, this transport is backend-agnostic; the backend can retrieve data either via direct pointer access (for C++ environments) or via DPI calls (for SystemVerilog simulators). This direct access capability allows vTen to approach the theoretical performance limit of native C++ testbenches.

Control Plane: Kernel-Granular Batch Execution. The control plane amortizes IPC latency through Kernel-Granular Batching. A Kernel Task encapsulates a complete execution unit: the target kernel instance, and the compiled Execution IR sequence. The host submits this atomic batch to the backend and the synchronization is managed via a simple lightweight handshake (Host start $\leftrightarrow$ Backend complete). This mechanism reduces the interaction overhead from linear with cycles ($O(C)$) to constant per kernel launch $O(1)$, preserving cycle-level debugging capability via BFMs.

3.4.3 Performance Modeling. To quantify the impact of the proposed architecture, we decompose the total verification latency (T_{total}) into four distinct stages: (1) **Tensor Serialization (T_{ser})**: The runtime overhead for configuration resolution and data serialization; (2) **Data Loading (T_{load})**: The transport latency for populating the shared memory buffer; (3) **Test Execution (T_{exec})**: The backend simulation time dominated by the DUT’s kernel execution; and (4) **Verification (T_{ver})**: The post-execution phase involving output deserialization and golden result comparison. The total latency is modeled as:

$$T_{total} = \sum_{i=1}^N (T_{ser} + T_{load} + T_{exec} + T_{ver}) \quad (1)$$

where N denotes the total number of kernel launches in the workload. In traditional cycle-driven frameworks (e.g., Cocotb), data transport is interleaved with execution steps, causing the combined overhead of signal driving and synchronization to scale linearly with execution cycles ($O(C)$). In contrast, vTen’s split-process architecture confines T_{ser} and T_{load} to the host-side runtime and utilizes

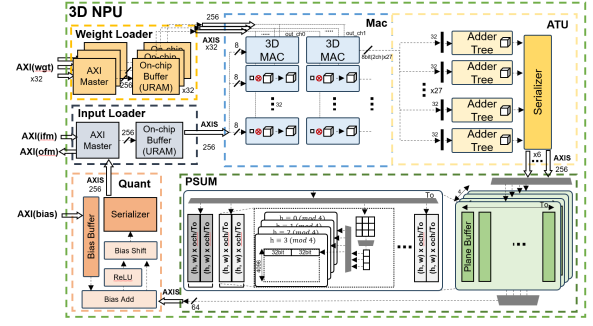


Figure 6: An Overview of the DUT architecture.

batching to render synchronization overhead negligible, allowing T_{exec} to dominate the total runtime ($T_{total} \approx T_{exec}$).

4 Evaluation

4.1 Experimental Setup

Design Under Test (DUT). We evaluate the performance of vTen against Cocotb on a custom NPU shown in Figure 6 for 3D segmentation models such as 3D U-Net [24]. Such a complicated DUT is chosen to analyze Cocotb’s overall VPI overhead associated with system integration of multiple submodule layers. The architecture comprises distributed IPs linked via AXI4-Stream, utilizing AXI4-Lite for configuration and AXI4 for tensor access (IFMs, Weights, OFMs). These interfaces connect to vTen or Cocotb BFMs during simulation. The design demonstrates production-scale complexity, utilizing 1,802 DSPs and 385 URAM blocks when synthesized on a Xilinx Alveo U280 FPGA at 125 MHz.

Workloads. We select Conv3D layers (kernel=3, stride=1) for workload evaluation, as their large tensor footprints heavily utilize the DUT’s interface bandwidth. We evaluate multiple test cases with increasing tensor sizes and NPU cycle counts, starting from $(C, D, H, W) = (32, 4, 4, 4)$ and doubling dimensions, to monitor overheads with diverse VPI count and payload sizes.

Per-Stage Latency Analysis. Following Section 3.4, we decompose simulation latency into: Tensor Serialization (T_{ser}), Data Loading (T_{load}), Test Execution (T_{exec}), and Verification (T_{ver}). We focus on T_{load} and T_{exec} , which directly expose architectural trade-offs, while T_{ser} and T_{ver} are framework-independent.

Environment. Experiments run on Ubuntu 20.04 with Intel Xeon Silver 4210R (40 cores, 2.40 GHz, 1 TB RAM) using Verilator as the simulation backend.

4.2 Abstraction Boundary Analysis

We evaluate five transport mechanisms in increasing overhead order: (1) **Native C++ (Theoretical Lower Bound)**: A cycle-accurate C++ testbench with internal data generation, serving as the theoretical latency lower bound. (2) **vTen (Pure C++)**: High-performance mode in which the backend reads Shared Memory (SHM) directly, bypassing SystemVerilog. (3) **vTen (SV-DPI)**: Compatibility mode for standard simulators where SV BFMs retrieve SHM data via DPI calls. (4) **File I/O**: Legacy SV testbench reading vectors from disk. (5) **VPI (Cocotb)**: Excluded from overhead analysis due to prohibitive latency ($> 100\times$).

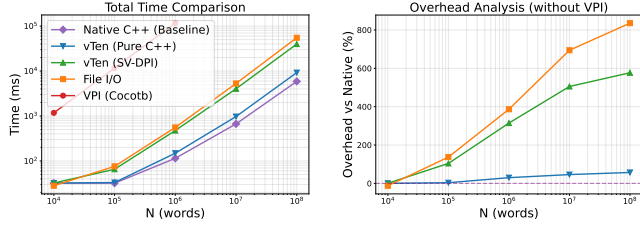


Figure 7: Scalability Analysis: (Left) Total simulation time vs. Tensor size (N). (Right) Overhead percentage relative to Native C++ execution.

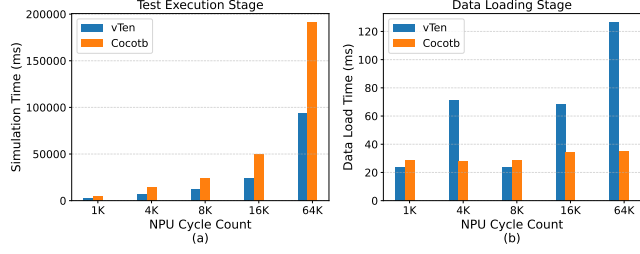


Figure 8: Latency Comparison between vTen and Cocotb: vTen (blue) achieves a 2 $\times$ performance increase over Cocotb (orange) in the Test Execution stage (a). vTen incurs a higher latency in the Data Loading stage (b) compared to Cocotb, though the overhead is minimal compared to (a).

Performance & Trade-off Analysis. Figure 7 illustrates the scalability as the tensor size (N) increases from 10^4 to 10^8 words. The vTen (Pure C++) backend demonstrates near-native efficiency, tracking the Native C++ lower bound with $<10\%$ overhead by completely eliminating the SystemVerilog boundary. In contrast, the overhead of the vTen (SV-DPI) backend increases linearly due to the inevitable context-switching cost ($SV \leftrightarrow C$), yet it still outperforms the legacy File I/O approach which suffers from prohibitive disk latency. By fundamentally decoupling data transport (Shared Memory) from control (Kernel-Granular Batching), vTen effectively eliminates these IPC bottlenecks, establishing itself as a scalable architecture for verifying data-intensive 3D workloads.

4.3 End-to-End Performance Analysis

This experiment evaluates the end-to-end performance on the 3D U-Net case study defined in Section 4.1. Figure 8(a) shows that vTen achieves a 2 $\times$ speedup in Test Execution (T_{exec}) compared to Cocotb across workloads ranging from 1K to 64K NPU cycles. Since the DUT execution itself is identical, this gain is attributed entirely to the elimination of VPI-induced context switching required for clock advancement. In Cocotb, the Python interpreter must wake up every cycle to drive the clock and signals, creating a dominant overhead that scales with simulation time ($O(C)$). vTen removes this bottleneck by offloading clock generation and batched transactions to the backend. This architectural shift effectively neutralizes the cumulative latency incurred by massive repetitive operations (Challenge 1).

While Figure 8(b) indicates higher loading latency (T_{load}) for vTen, this results from *front-loading* data transfer to Shared Memory, whereas Cocotb defers signal driving to the execution phase. This one-time setup effectively decouples transport from execution,

enabling the performance gains demonstrated in Figure 8(a). Crucially, this upfront cost remains negligible ($<1\%$ of total latency), ensuring that transport does not bottleneck the overall simulation.

4.4 Verification Productivity Analysis

Addressing Challenge 2 (System Integration) with heterogeneous IPs (e.g., DMA, MAC), vTen achieves a 92.2% reduction in verification-specific code (28 vs. 360 lines). By encapsulating golden references within Kernel Objects, vTen facilitates hierarchical composition, allowing users to simply aggregate existing unit specifications instead of rewriting top-level logic. Consequently, this approach eliminates 242 lines of manual serialization and AXI4 sequencing required in Cocotb. Including the shared golden model, vTen demonstrates a 60.3% overall reduction (261 vs. 658 lines), proving that Kernel Abstraction resolves integration bottlenecks via modular reuse.

5 Related Work

Existing hardware verification frameworks follow three approaches. Simulation acceleration includes hardware-based solutions (FireSim[13], DiffTest-H[23], ASH[5]) that leverage FPGA/emulator platforms for faster simulation, and software optimizations (Verilua[4]) that improve simulator performance. Design-verification languages (Chisel[3], PyMTL3[10]) integrate verification capabilities directly into hardware construction languages, enabling unified design and test flows. Portable verification frameworks (cocotb[20], fault[21]) provide language-agnostic interfaces for test reuse across different simulators and formal tools. However, these approaches face fundamental limitations: hardware acceleration is primarily suited for SoC-level verification with significant setup overhead, design-specific languages create fragmented toolchains, and portable frameworks sacrifice performance for abstraction. These limitations are particularly acute for AI accelerator verification, which requires handling massive tensor datasets and complex data serialization—motivating vTen’s compiled verification paradigm that eliminates per-transaction overhead while enhancing verification productivity.

6 Conclusion

In this paper, we propose vTen, a data-centric verification framework designed to bridge the semantic gap in Domain-Specific Accelerator (DSA) verification. By decoupling verification intent from execution mechanics through declarative DSL and kernel-granular batching, vTen effectively eliminates the synchronization bottlenecks found in traditional methodologies. Evaluation on a 3D U-Net accelerator demonstrates that vTen achieves a 2x performance improvement in test execution and a 60.3% reduction in code complexity compared to Cocotb[20]. These evaluation results highlight vTen’s capability to provide a scalable, backend-agnostic verification solution, lowering entry barriers and accelerating agile development for next-generation AI hardware.

Acknowledgments

This research was supported in part by the ITRC (Information Technology Research Center) support program (IITP-2026-2020-0-01461) supervised by the IITP, funded by the MSIT (Ministry of Science and ICT), Korea, and in part by the Graduate School of Artificial Intelligence Semiconductor (IITP-2026-RS-2023-00256081)

grant funded by the MSIT. The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

References

- [1] 2020. IEEE Standard for Universal Verification Methodology Language Reference Manual. *IEEE Std 1800.2-2020 (Revision of IEEE Std 1800.2-2017)* (2020), 1–458. doi:10.1109/IEEESTD.2020.9195920
- [2] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: a system for Large-Scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.
- [3] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzynnek, and Krste Asanović. 2012. Chisel: constructing hardware in a scala embedded language. In *Proceedings of the 49th annual design automation conference*. 1216–1225.
- [4] Ye Cai, Chuyu Zheng, Wei He, and Dan Tang. 2025. Verilua: An Open Source Versatile Framework for Efficient Hardware Verification and Analysis Using LuaJIT. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–2.
- [5] Fares Elsabbagh, Shabnam Sheikhha, Victor A Ying, Quan M Nguyen, Joel S Emer, and Daniel Sanchez. 2023. Accelerating rtl simulation with hardware-software co-design. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 153–166.
- [6] Harry D. Foster. 2024. Wilson Research Group IC/ASIC functional verification trend report. <https://resources.sw.siemens.com/en-US/white-paper-2024-wilson-research-group-ic-asic-functional-verification-trend-report/>
- [7] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, et al. 2021. Gemini: Enabling systematic deep-learning architecture evaluation via full-stack integration. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 769–774.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *CoRR* abs/1512.03385 (2015). arXiv:1512.03385 <http://arxiv.org/abs/1512.03385>
- [9] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. 2022. DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–17. doi:10.1109/HCS55958.2022.9895626
- [10] Shunning Jiang, Peitian Pan, Yanghui Ou, and Christopher Batten. 2020. PyMTL3: A python framework for open-source hardware modeling, generation, simulation, and verification. *IEEE Micro* 40, 4 (2020), 58–66.
- [11] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [12] Rachid Karami, Sheng-Chun Kao, and Hyoukjun Kwon. 2025. Understanding the Performance Horizon of the Latest ML Workloads with NonGEMM Workloads. arXiv:2404.11788 [cs.AR] <https://arxiv.org/abs/2404.11788>
- [13] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, et al. 2018. FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.
- [14] Minsik Kim, Kyoungseok Oh, Youngmook Cho, Hojin Seo, Xuan Truong Nguyen, and Hyuk-Jae Lee. 2024. A Low-Latency FPGA Accelerator for YOLOv3-Tiny With Flexible Layerwise Mapping and Dataflow. *IEEE Transactions on Circuits and Systems I: Regular Papers* 71, 3 (2024), 1158–1171. doi:10.1109/TCSI.2023.3335949
- [15] Keehyuk Lee, Mincheol Cha, Soosung Kim, Xuan Truong Nguyen, and Hyuk-Jae Lee. 2025. A Winograd-Convolution-Based Accelerator on FPGA for Real-Time Object Detection with Effective On-Chip Buffer Access Patterns. In *2025 IEEE International Conference on Consumer Electronics (ICCE)*. 1–5. doi:10.1109/ICCE63647.2025.10930191
- [16] Duy Thanh Nguyen, Hyeonseung Je, Tuan Nghia Nguyen, Soojung Ryu, Kyujoong Lee, and Hyuk-Jae Lee. 2022. ShortcutFusion: From Tensorflow to FPGA-Based Accelerator With a Reuse-Aware Memory Allocation for Shortcut Data. *IEEE Transactions on Circuits and Systems I: Regular Papers* 69, 6 (2022), 2477–2489. doi:10.1109/TCSI.2022.3153288
- [17] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [18] Joseph Redmon and Ali Farhadi. 2018. YOLOv3: An Incremental Improvement. *CoRR* abs/1804.02767 (2018). arXiv:1804.02767 <http://arxiv.org/abs/1804.02767>
- [19] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. *CoRR* abs/1505.04597 (2015). arXiv:1505.04597 <http://arxiv.org/abs/1505.04597>
- [20] Benjamin John Rosser. 2018. Cocotb: a Python-based digital logic verification framework. In *Micro-electronics Section seminar. CERN, Geneva, Switzerland*.
- [21] Lenny Truong, Steven Herbst, Rajsekhar Setaluri, Makai Mann, Ross Daly, Keyi Zhang, Caleb Donovick, Daniel Stanley, Mark Horowitz, Clark Barrett, et al. 2020. fault: A python embedded domain-specific language for metaprogramming portable hardware verification components. In *International Conference on Computer Aided Verification*. Springer, 403–414.
- [22] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 97–110.
- [23] Kunlin You, Yinan Xu, Kehan Feng, Luoshan Cai, Yaoyang Zhou, and Yungang Bao. 2025. DiffTest-H: Toward Semantic-Aware Communication in Hardware-Accelerated Processor Verification. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1462–1476.
- [24] Özgün Çiçek, Ahmed Abdulkadir, Soeren S. Lienkamp, Thomas Brox, and Olaf Ronneberger. 2016. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. arXiv:1606.06650 [cs.CV] <https://arxiv.org/abs/1606.06650>